



IMT Atlantique

Bretagne-Pays de la Loire
École Mines-Télécom



An Online Learning-based Metaheuristic for Solving Combinatorial Optimization Problems

Maryam KARIMI MAMAGHAN

Mehrdad MOHAMMADI

Bastien PASDELOUP

Romain BILLOT

Patrick MEYER

CONTENT

1. INTRODUCTION

- 1.1 Preliminaries
- 1.2 Iterated Local Search (ILS)
- 1.3 Reinforcement Learning: *Q*-Learning

2. PROPOSED ALGORITHM

- 2.1 Proposed *Q*-ILS Algorithm
- 2.2 Case Study: TSP

3. RESULTS

- 3.1 Experimental Design: TSPLIB Instances
- 3.2 Numerical Results: Berlin52
- 3.3 Numerical Results: pcb442

4. CONCLUSION & FUTURE RESEARCH



IMT Atlantique
Bretagne-Pays de la Loire
École Mines-Télécom



1.1 Preliminaries

- **Metaheuristics (MHs)**
 - Provide acceptable solutions in reasonable computation time
 - Good substitutes for exact algorithms
- **Machine Learning (ML)**
 - learn from a dataset and are capable of making prediction
 - Good approach for solving Combinatorial Optimization Problems (COPs)
- **Integration of ML techniques with Metaheuristics**
 - Emerging research field (Jourdan et al., 2006- Zhang et al., 2011- Talbi, 2016- Song et al., 2019)
 - Improve the performance of MHs
- **This study**
 - Online learning, Reinforcement learning
 - Iterated local search
 - Reactive Search Optimization
 - Traveling Salesman Problem (TSP)

1.1 Iterated Local Search (ILS)

- ILS algorithm procedure

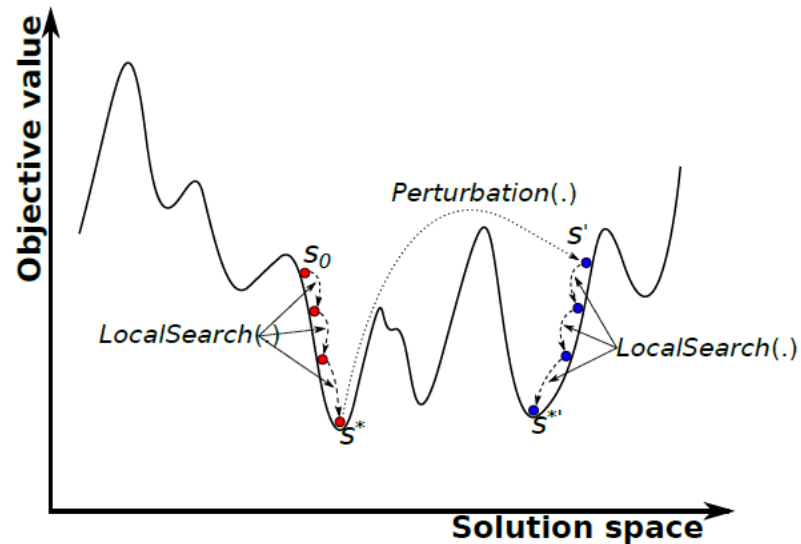
Step 1. Generate Initial solution

Step 2. LocalSearch(.) to obtain a local optimal solution

Step 3. AcceptanceCriterion(.) to check whether the current local optimal solution is accepted

Step 4. Perturbation(.) to help the search escape the local optimal solution

- Single *LocalSearch(.)* operator

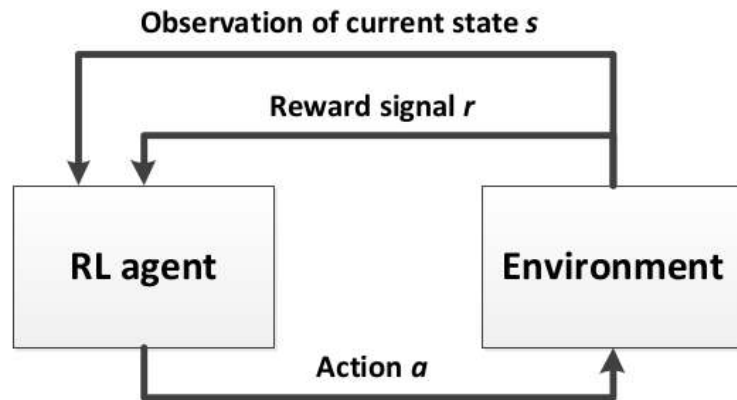


1.2 Reinforcement Learning: Q-Learning

- The Q-Learning algorithm
 - A model-free reinforcement learning algorithm

$$Q(s, a) = Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

- $Q(s, a)$: the value associated with the state-action pair (s, a)
- s : the current state
- a : the action performed in state s
- r : the reinforcement signal (return or reward) received after performing a in s
- s' : the next state
- $0 \leq \gamma < 1$: the discount factor
- $0 \leq \alpha < 1$: the learning rate



Wauters, et al., 2013. Boosting metaheuristic search using reinforcement learning. In Hybrid Metaheuristics (pp. 433-452). Springer, Berlin, Heidelberg.

2.1 Proposed Q-ILS Algorithm

- ILS with multiple local search operators
- Operator selection: Randomly (R-ILS) vs Intelligently (Q-ILS)
- The Q-Learning algorithm

$$Q(s, a) = Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

- State: sequence of last k local search operators ($k=1$)
- Action: a local search operator
- Reward (punishment): proportional improvement (deterioration)
- Q -value matrix

$$Q(s, a) = \begin{matrix} & \text{state/action} & op_1 & \dots & op_A \\ \begin{matrix} op_1 \\ \vdots \\ op_S \end{matrix} & & \begin{bmatrix} \dots & \dots & \dots \\ \vdots & \ddots & \vdots \\ \dots & \dots & \dots \end{bmatrix} \end{matrix}$$

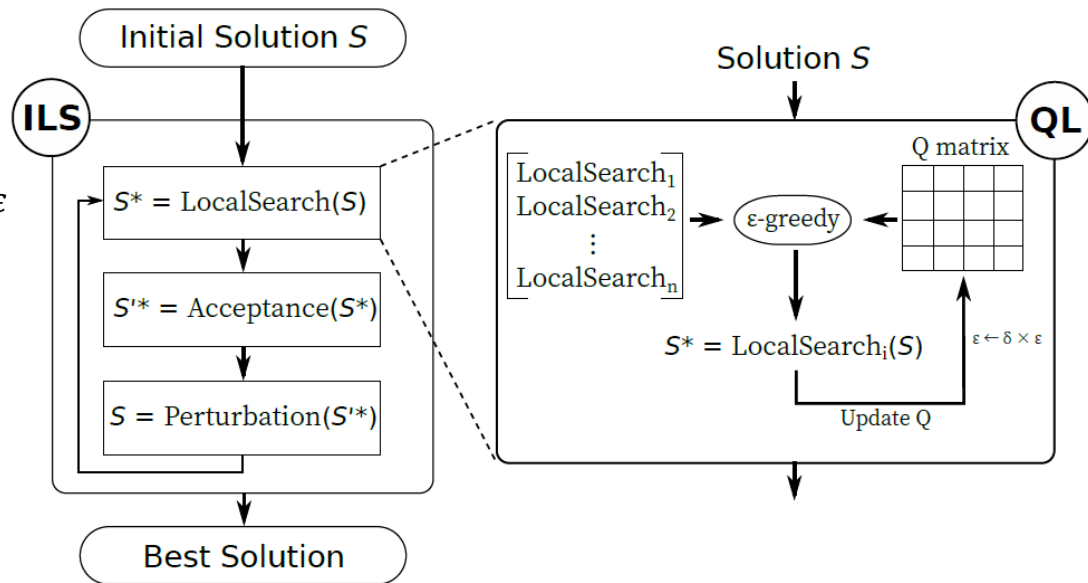
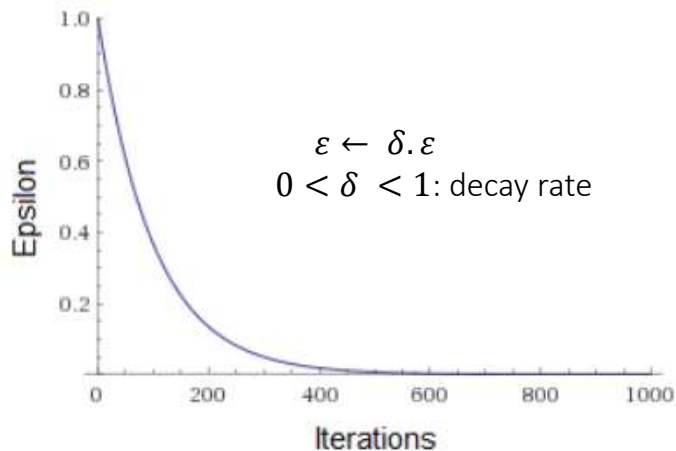
CHAPTER 2: PROPOSED ALGORITHM

7

2.1 Proposed Q-ILS Algorithm

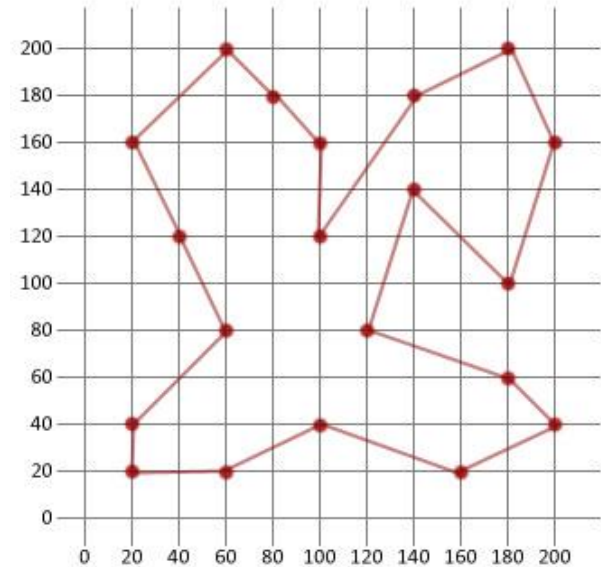
- Diversification and Intensification
 - ε -greedy strategy with ε -decay

$$\pi(s) = \begin{cases} a_{\text{random}} & , \text{probability } \varepsilon \\ \operatorname{argmax}_a Q(s, a), & \text{probability } 1 - \varepsilon \end{cases}$$



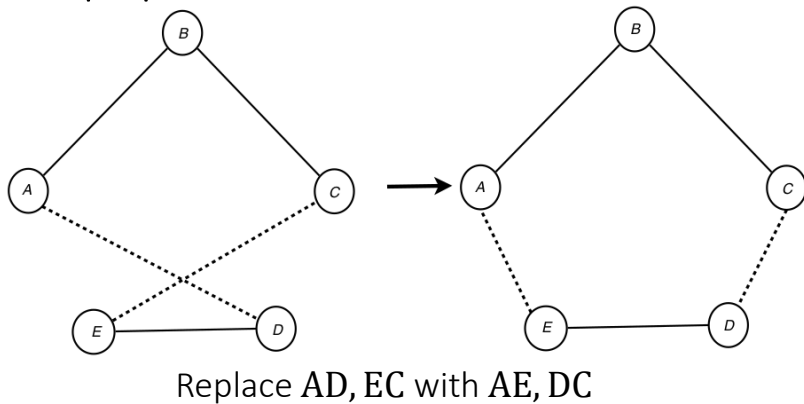
2.2 Case Study: TSP

- Problem
 - Traveling Salesman Problem (TSP) can be defined by means of a weighted graph $G = (V, E)$ in which V is a set of vertices and E is a set of edges. Each vertex represents a city and each edge connects 2 vertices. The weight of each edge is the distance between two cities.
- Goal
 - Finding the Hamiltonian cycle of minimum total length
- $LocalSearch(.) = \{2opt, Swap, Insertion\}$
- $Perturbation(.) = double_bridge$

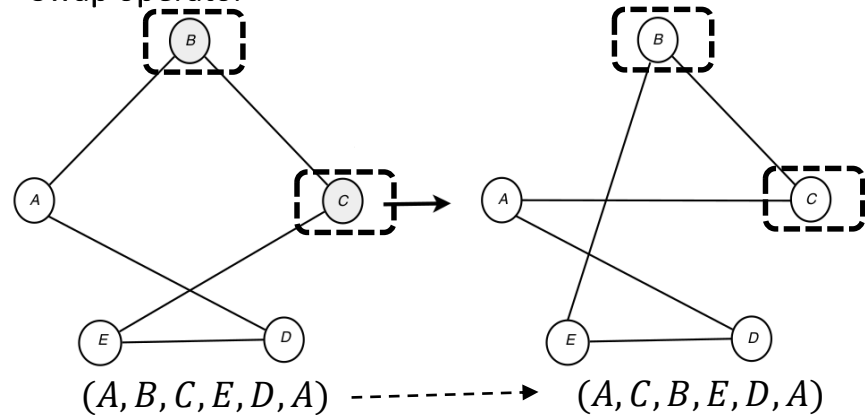


2.2 Case Study: TSP

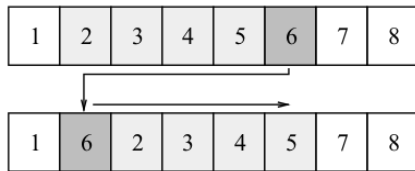
- 2-opt operator



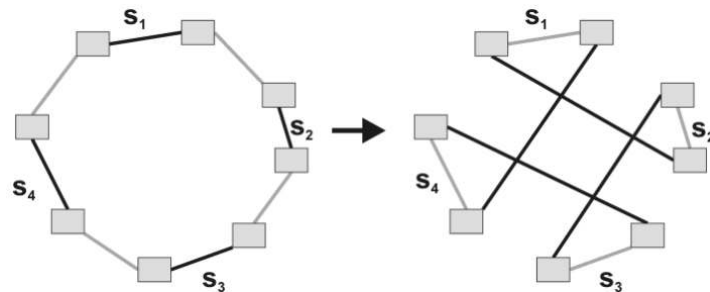
- Swap operator



- Insertion operator



- Double-bridge operator

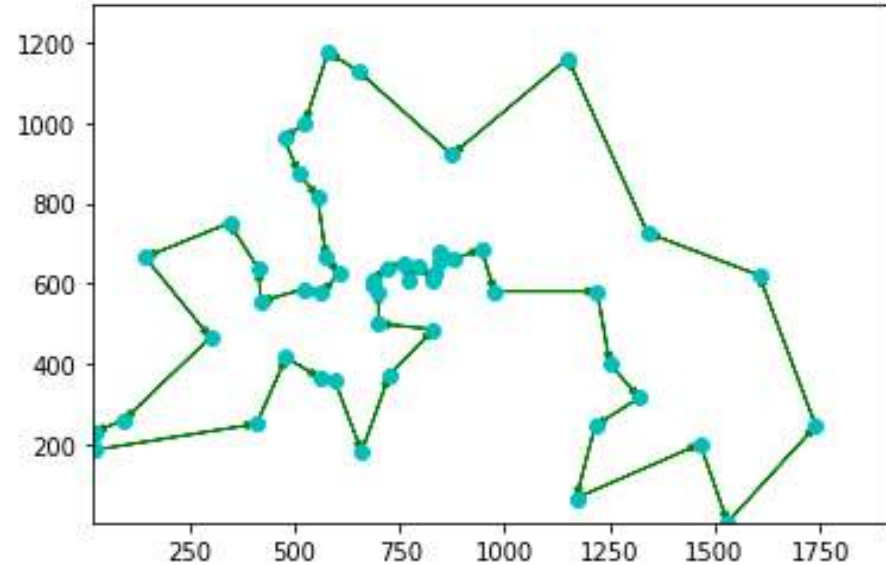


3.1 Experimental Design: TSPLIB Instances

- TSPLIB datasets
 - **Berlin52** (52 nodes)
 - **Pcb442** (442 nodes)
- Parameter setting
 - $\varepsilon \in]0,1], \Delta\varepsilon = 0.1$
 - $\delta \in [0.9,1], \Delta\delta = 0.01$
 - $\alpha \in]0,1], \Delta\alpha = 0.1$
 - $\gamma \in]0,1], \Delta\gamma = 0.1$
- Tuned parameters
 - *Iteration* = 400 (Berlin52)
 - *Iteration* = 1000 (Pcb442)
 - $\varepsilon = 0.7$
 - $\delta = 0.7$
 - $\alpha = 0.4$
 - $\gamma = 0.4$

3.2 Numerical Results: Berlin52

- Best found & Optimal tour
- Best time = 2 s

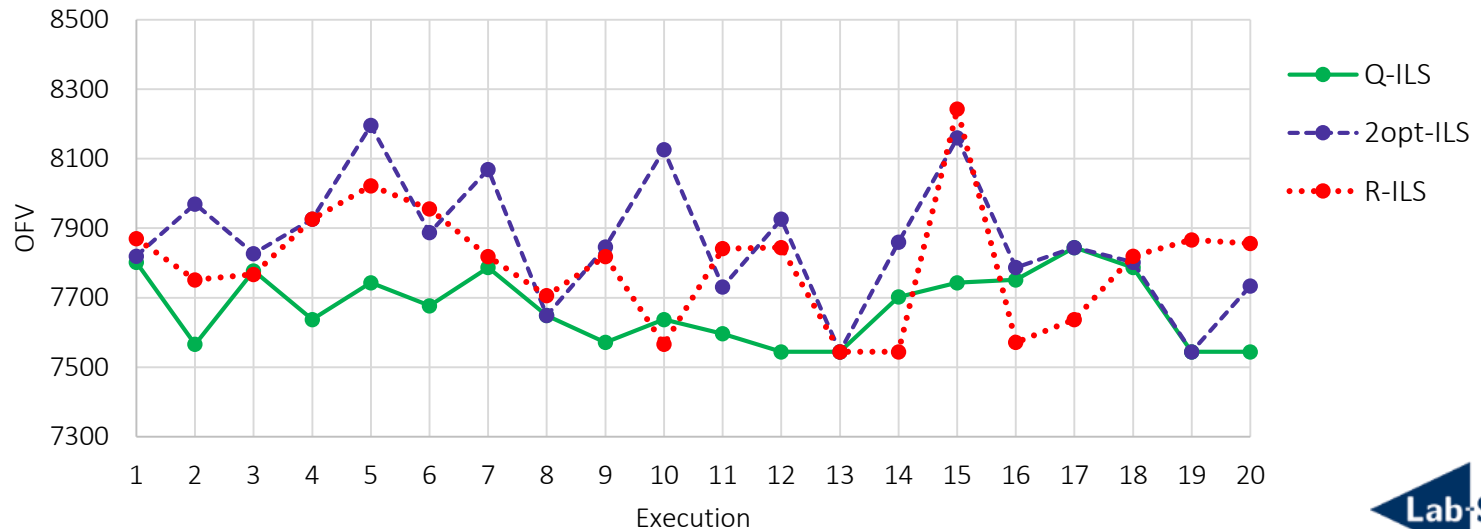


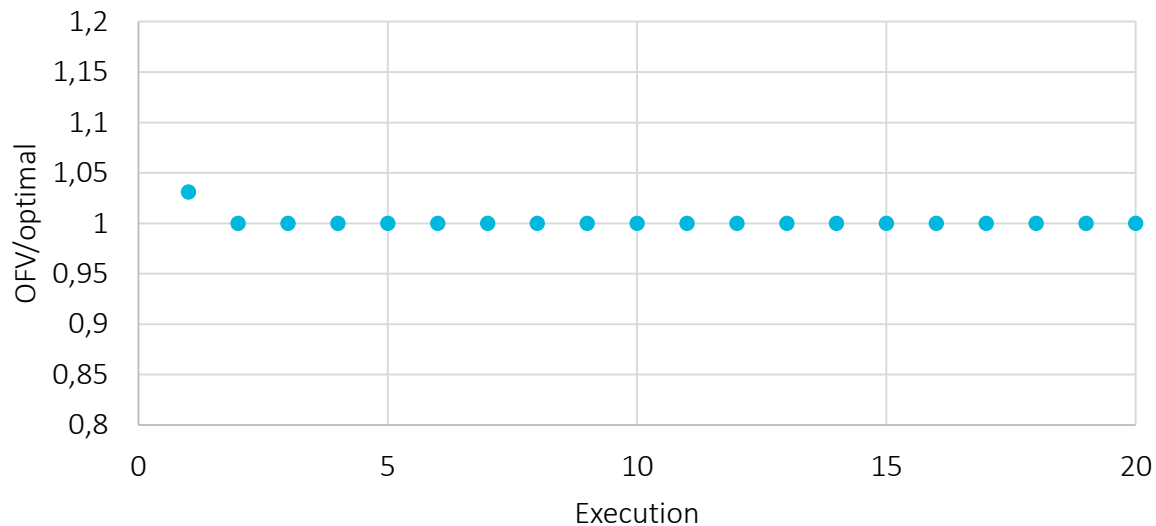
CHAPTER 3: RESULTS

3.2 Numerical Results: Berlin52

12

	Algorithm								
	Q-ILS			2opt-ILS			R-ILS		
	OFV	Time (s)	Gap (%)	OFV	Time (s)	Gap (%)	OFV	Time (s)	Gap (%)
μ	7672.44	4.21	1.7	7862.26	4.81	4.2	7798.40	6.78	3.4
σ	101.74	0.25		181.49	0.28		174.59	0.34	





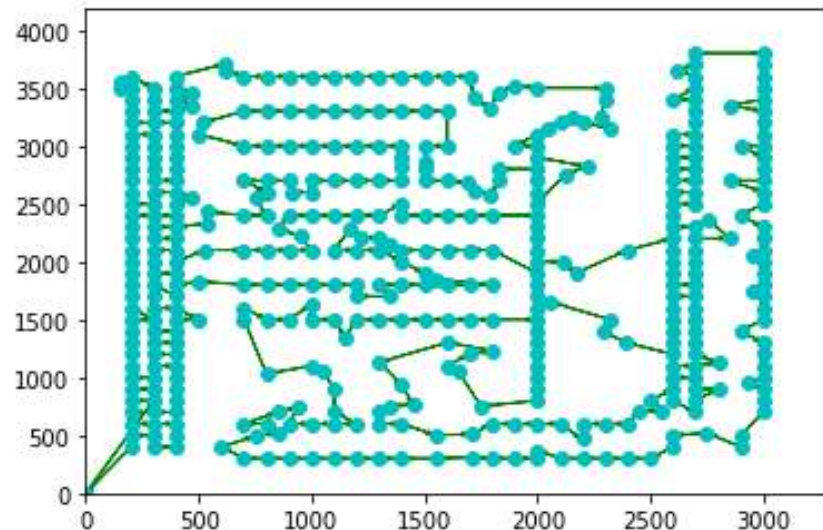
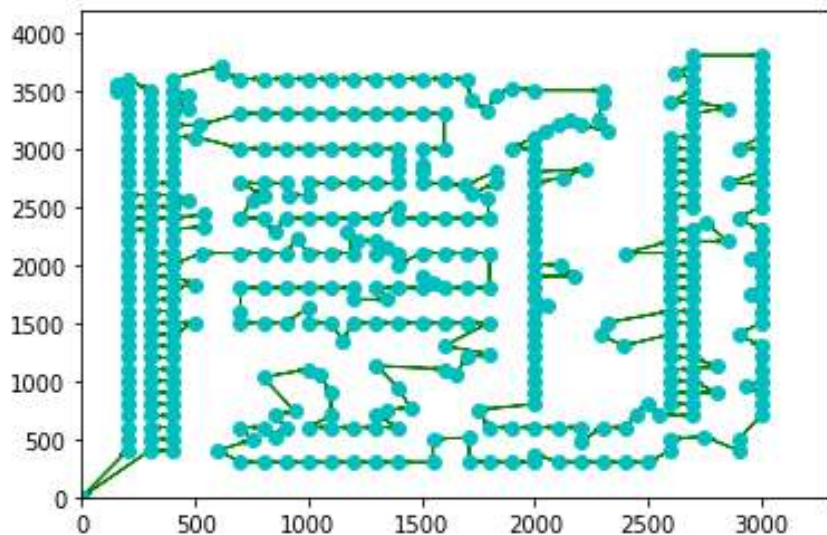
CHAPTER 3: RESULTS

3.3 Numerical Results: pcb442

14

- Optimal tour
- OFV = 50783

- Best found tour
- OFV = 51941 (Gap = 2.2%)
- Time = 360 s

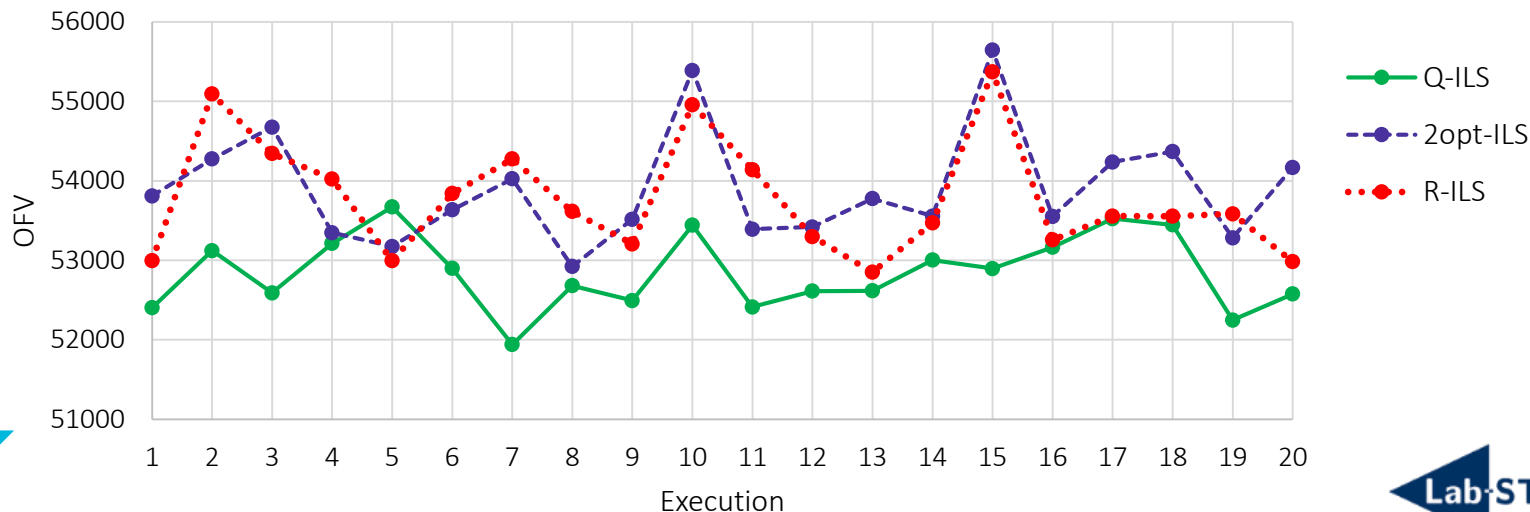


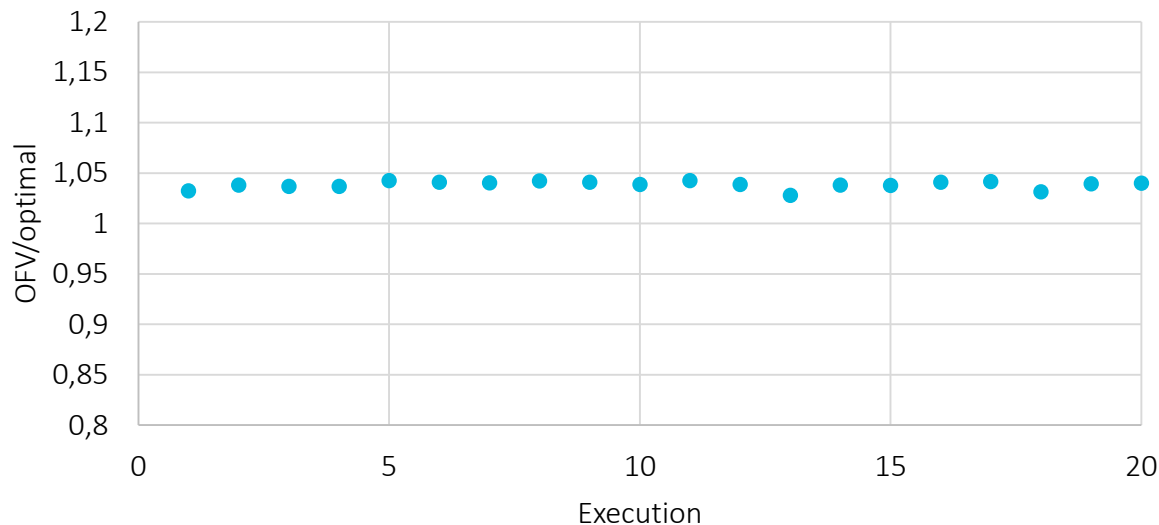
CHAPTER 3: RESULTS

3.3 Numerical Results: pcb442

15

	Algorithm								
	Q-ILS			2opt-ILS			R-ILS		
	OFV	Time (s)	Gap (%)	OFV	Time (s)	Gap (%)	OFV	Time (s)	Gap (%)
μ	52848.08	356.50	4.1	53908.77	415.55	6.2	53771.02	430.98	5.9
σ	467.96	11.82		709.63	16.74		733.64	27.69	





- Conclusion
 - **Reactive** search optimization: **ILS + Q-Learning**
 - Berlin52 and Pcb442 datasets from TSPLIB
 - Q-ILS outperforms the 2opt-ILS and R-ILS in both OFV (**2.5% & 1.6%**) and Time (**14% & 61%**)
- Future Research
 - Test our proposed approach on **different datasets/ problems**
 - Adding **more local search operators** to the proposed algorithm
 - Taking into account the **past sequence of local search operators** ($k > 1$)
 - **Intelligent** perturbation operator selection

Thanks for your
attention

- Dos Santos, J. P. Q., de Melo, J. D., Neto, A. D. D., & Aloise, D. (2014). Reactive search strategies using reinforcement learning, local search algorithms and variable neighborhood search. *Expert Systems with Applications*, 41(10), 4939-4949.
- Jourdan, L., Dhaenens, C., & Talbi, E. G. (2006, October). Using datamining techniques to help metaheuristics: A short survey. In *International Workshop on Hybrid Metaheuristics* (pp. 57-69). Springer, Berlin, Heidelberg.
- Lourenço, H. R., Martin, O. C., & Stützle, T. (2003). Iterated local search. In *Handbook of metaheuristics* (pp. 320-353). Springer, Boston, MA.
- Song, H., Triguero, I., & Özcan, E. (2019). A review on the self and dual interactions between machine learning and optimisation. *Progress in Artificial Intelligence*, 8(2), 143-165.
- Talbi, E. G. (2009). *Metaheuristics: from design to implementation* (Vol. 74). John Wiley & Sons.
- Talbi, E. G. (2016). Combining metaheuristics with mathematical programming, constraint programming and machine learning. *Annals of Operations Research*, 240(1), 171-215.
- TSPLIB: <http://elib.zib.de/pub/mp-testdata/tsp/tsplib/tsplib.html>
- Wauters, T., Verbeeck, K., De Causmaecker, P., & Berghe, G. V. (2013). Boosting metaheuristic search using reinforcement learning. In *Hybrid Metaheuristics* (pp. 433-452). Springer, Berlin, Heidelberg
- Zhang, J., Zhan, Z. H., Lin, Y., Chen, N., Gong, Y. J., Zhong, J. H., ... & Shi, Y. H. (2011). Evolutionary computation meets machine learning: A survey. *IEEE Computational Intelligence Magazine*, 6(4), 68-75.

Iterated Local search (ILS) pseudo code

Iterated Local Search - ILS

Generate an initial solution s_0

$s^* = \text{LocalSearch}(s_0)$

// Initial solution is intensified to reach a local optimum

$s_{best} = s^*$

// Best solution is archived

While stopping criterion not reached **Do**

$s' = \text{Perturbation}(s^*)$

// Local optimum solution is diversified

$s^{*'} = \text{LocalSearch}(s')$

// New local solution is found

$s^* = \text{AcceptanceCriterion}(s^*, s^{*'}, s_{best})$

// New local optimum is conditionally accepted

EndWhile

Proposed Q-ILS Algorithm pseudo code

The proposed Q-ILS algorithm

```

1   $t \leftarrow 0$ 
2   $S_0 \leftarrow \text{Generate Initial Solution}$ 
3   $\text{available} \leftarrow \text{Set of all possible actions}$  // action equals to operators
4   $\text{state} \leftarrow \text{select local search (available)}$ 
5   $Q \leftarrow [0]$  // Q matrix of Q Learning
6   $S_0 \leftarrow \text{local search}(S_0)$ 
7   $S^* \leftarrow S_0$ 
8  While  $t \leq \text{Max Iteration}$  Do
9       $\text{action} \leftarrow \text{Select local search (available)}$ 
10      $S_0 \leftarrow \text{local search}(S_0, \text{action})$ 
11     If  $S_0$  better than  $S^*$  Then
12          $S^* \leftarrow S_0$ 
13          $\text{update } Q(\text{state}, \text{action})$  // adaptive reward
14          $\text{state} \leftarrow \text{action}$ 
15     Else
16          $\text{update } Q(\text{state}, \text{action})$  // adaptive punishment
17          $\text{available} \leftarrow \text{available} - \text{action}$ 
18     EndIf
19     If  $\text{available} = \emptyset$  // no action remains
20          $S_0 \leftarrow \text{Perturbation}(S_0)$ 
21          $\text{available} \leftarrow \text{Set of all possible actions}$ 
22     EndIf
23      $t \leftarrow t + 1$ 
EndWhile

```
